

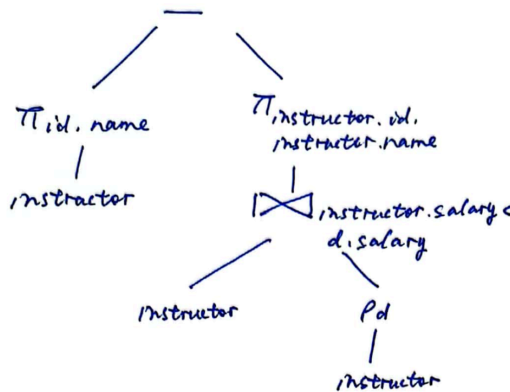
Sample solution to Assignment (I)

Problem 1 (10 points)

Write relational algebra expressions for the following queries, utilizing the university schema outlined in the textbook. A brief description of the university schema can be found on pp. 37-44 and a sample database can be found on pp. 1292-1298. Prior to presenting your relational algebra expressions, Please draw the corresponding relational algebra expression tree for each query.

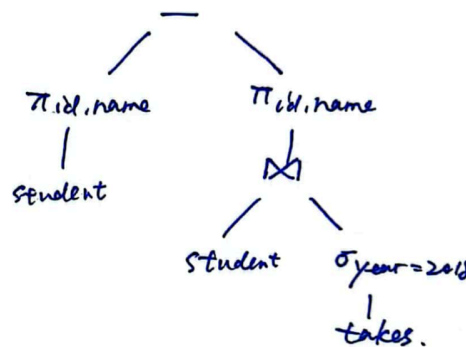
1. Retrieve the ID and name of each instructor with the highest salary.

$$\Pi_{id, name}(instructor) - \Pi_{instructor.id, instructor.name}((instructor) \bowtie_{instructor.salary < d.salary} \rho_d(instructor))$$



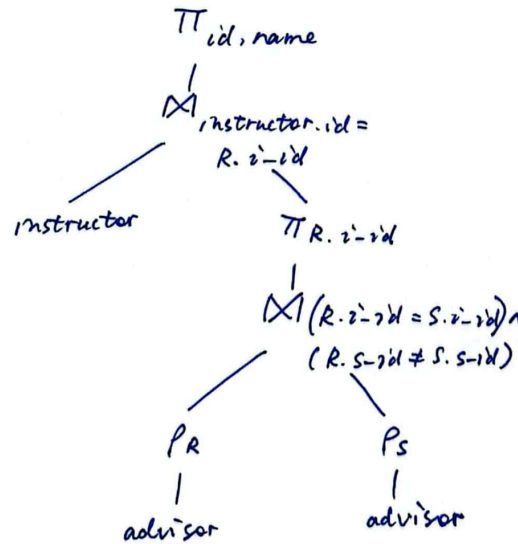
2. Retrieve the ID and name of each student who has not taken any course section in 2018.

$$\Pi_{id, name}(student) - \Pi_{id, name}(student \bowtie (\sigma_{year=2018}(takes)))$$



3. Retrieve the ID and name of each instructor who has advised more than one student. It's noteworthy that the table `advisor(s_id, i_id)` stores the advisor-advisee relationship where `i_id` (resp. `s_id`) is a foreign key referencing the table `instructor` (resp. `student`).

$$\Pi_{id, name}(instructor \bowtie_{instructor.id=R.i_id} (\Pi_{R.i_id}(\rho_R(advisor) \bowtie_{(R.i_id=S.i_id) \wedge (R.s_id \neq S.s_id)} \rho_S(advisor)))$$



Problem 2 (10 points)

Let R be a relation with the schema $(A_1, \dots, A_n, B_1, \dots, B_m)$, and let S be a relation with the schema $(B_1, \dots, B_m)$. In other words, the attributes of S form a subset of the attributes of R . The **quotient** of R by S , denoted as $R \div S$, comprises tuples t over attributes $A_1, \dots, A_n$ such that for every tuple s in S , the tuple ts which includes the components of t for $A_1, \dots, A_n$ and the components of s for $B_1, \dots, B_m$, belongs to R . Below is a simple illustrative example.

A	B ₁	B ₂
1	2	3
1	4	5
2	2	3
3	3	4

Table 2: $R(A, B_1, B_2)$

B ₁	B ₂
2	3
4	5

Table 3: $S(B_1, B_2)$

A
1

Table 4: $R \div S$

Please write a relational algebra expression that is equivalent to $R \div S$, utilizing only the operations introduced in our lecture.

$$\Pi_{A_1, \dots, A_n}(R) - \Pi_{A_1, \dots, A_n}(\Pi_{A_1, \dots, A_n}(R) \times S - R)$$

Problem 3 (SQL Lab, 80 points)

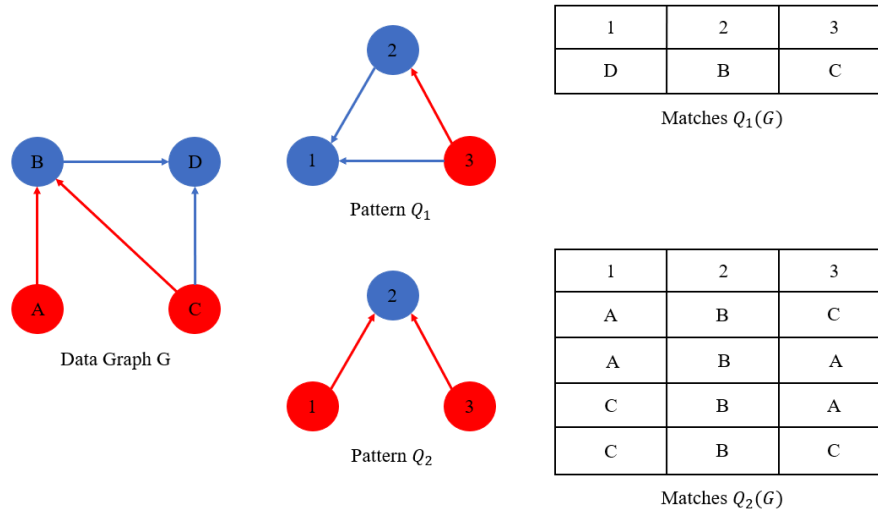
Background

Graph pattern match is a fundamental problem in graph computing with widespread applications in fields like financial fraud detection, social recommendations, computer vision, and bioinformatics.

We first briefly introduce the concepts of graph pattern matching. Let $G = (V, E)$ be a data graph and $Q = (V_Q, E_Q)$ be a pattern graph. Let $L_V : V \cup V_Q \rightarrow L_1$ be a node labeling function that assigns each graph/pattern node a label in L_1 . Additionally, let $L_E : E \cup E_Q \rightarrow L_2$ be an edge labeling function that defines an edge label in L_2 for each edge in both G and Q . A match of Q in G is a mapping $m : V_Q \rightarrow V$ such that (1) $L_v(u) = L_v(m(u))$ for each pattern node u , (2) for every pattern edge $(u, v) \in E_Q$ with label $L_E(u, v)$, there exists a data edge $(m(u), m(v))$ in G with the same label $L_E(u, v)$.

Given a data graph G and a pattern graph Q , graph pattern matching aims to compute $Q(G)$ which consists of all the matches of Q in G .

Below is a toy example to help you understand these concepts. Let G be a data graph with four nodes and consider two simple patterns: a triangle pattern Q_1 and a path pattern Q_2 . We use colors to represent the labels, i.e., $L_1 = L_2 = \{\text{red}, \text{blue}\}$.



It is not hard to verify that for the triangle pattern Q_1 , there is only one match

$$\{1 \rightarrow D, 2 \rightarrow B, 3 \rightarrow C\}$$

in G . Instead, for the path pattern Q_2 , its match result consists of four matches. Note that we adopt the **homomorphism** semantics for pattern matching. Thus it is feasible to map two different pattern nodes to the same data node. For example, one match in $Q_2(G)$ is

$$\{1 \rightarrow A, 2 \rightarrow B, 3 \rightarrow A\}$$

where both pattern node 1 and pattern node 3 are mapped to the data node A .

Last semester, your teaching assistants (TAs) participated in a **graph pattern matching competition** and won **second prize**. Their task was to identify suspicious behavior through pattern matching in a financial dataset. They were required to develop pattern-matching algorithms using a vertex-centric programming framework, a well-known approach in large-scale, distributed graph processing. You can consult the two papers listed below if you are curious about these concepts and techniques. (For the sake of this assignment, you may safely skip these materials.)

- [Pregel: a system for large-scale graph processing](#)
- [Thinking Like a Vertex: A Survey of Vertex-Centric Frameworks for Large-Scale Distributed Graph Processing](#)

To validate their implementation, for each pattern matching task, the TAs wrote a SQL query and executed it on a relational DBMS, thereby obtaining accurate benchmark results for reference. Indeed, SQL's features are robust enough to express such pattern matching queries effectively.

As a practical exercise following your SQL learning, your task will mirror that of the TAs: to write SQL queries for graph pattern matching tasks. To reduce your workload and let you concentrate on SQL queries, the TAs have already transformed the competition's graph data into a relational format. Your first task is to import a file named `data.sql` into PostgreSQL (see [Submission section](#) below).

file `data.sql` contains the schema definition of the dataset and a small subset sampled from the benchmark dataset. Afterward, you can write and test your SQL queries.

Datasets

We use the *FinBench (Financial Benchmark)* dataset in our SQL Lab as our TAs have done in their competition before. FinBench is a benchmark characterized by special data and query patterns in the financial industry to evaluate graph database systems in financial scenarios.

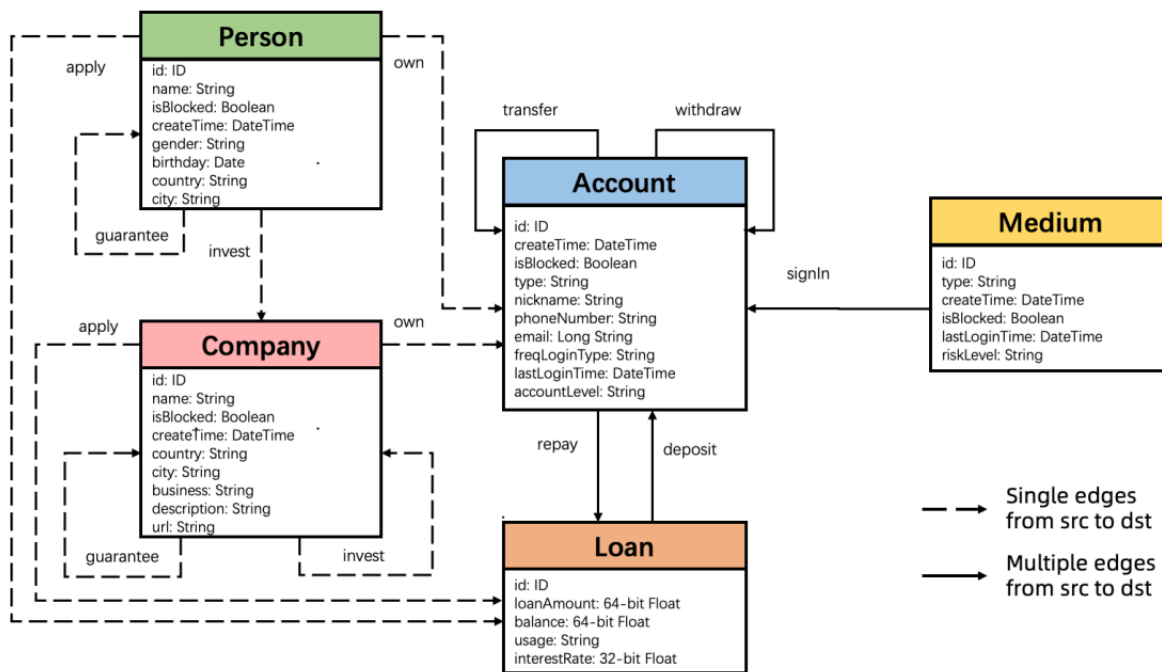


Fig 3.1: The LDBC FinBench data schema

Fig 3.1 shows the data schema in UML. The schema defines the structure of the data used in the benchmark in terms of entities and their relations. The data represents a snapshot of the activity in several financial scenarios during a period. The schema specifies different entities, their attributes, and their relations. In our SQL Lab, only 3 entities and 5 relations are used. The entities are **Account**, **Person** and **Loan**. The relations are **AccountTransferAccount**, **LoanDepositAccount**, **PersonApplyLoan**, **PersonGuaranteePerson** and **PersonOwnAccount**. In Fig 3.1, entities are depicted as rectangles, and relations are shown as edges connecting two entities.

Below is a concise description of the entities and relations that will be used in your SQL Lab.

1. **Account:** An account entity in real-world financial systems, which is registered and owned by a person or a company. It includes many types such as personalDeposit, personalCredit, etc. It can deal with other accounts. Table 3.1 shows the attributes of an account entity.

Attribute	Type	Description
id	ID	The identifier of the account.
createTime	DateTime	The time when the account is created.
isBlocked	Boolean	If the account is blocked or concerned in systems.
type	String	The type of the account.
nickname	String	The nickname of the account.
phoneNumber	String	The phone number of the account.
email	String	The email of the account.
freqLoginType	String	The frequent login type of the account.
lastLoginTime	DateTime	The last login time of the account.
accountLevel	String	The level of the account.

Table 3.1: Attributes of Account

In our SQL Lab, an entity is represented by a table. For instance, we introduce a table named `Account` for the account entities. The schema definition for the `Account` is shown below.

```

1 CREATE TABLE public.account (
2     accountid bigint,
3     createtime timestamp without time zone,
4     isblocked boolean,
5     accounttype character varying,
6     nickname character varying,
7     phonenumber character varying,
8     email character varying,
9     freqlogintype character varying,
10    lastlogintime bigint,
11    accountlevel character varying
12 );

```

2. **AccountTransferAccount:** The relation records fund transfers between account, i.e., the self-loop edge labeled `transfer` in Fig. 3.1. Its attributes are listed below.

Attribute	Type	Description
timestamp	DateTime	The time when transfer issues.
amount	64-bit Float	The amount of the transfer.
ordernumber	String	The order number of the transfer.
comment	String	The comment of the transfer.
payType	String	The pay type of the transfer.
goodsType	String	The goods type of the transfer.

Table 3.2: Attributes of AccountTransferAccount

In our SQL Lab, a relation between two entities is also represented by a table. We define a table named `AccountTransferAccount` to record the information between accounts. The schema definition for this table can be found in `data.sql`, as shown below.

```

1 CREATE TABLE public.accounttransferaccount (
2     fromid bigint,
3     toid bigint,
4     amount double precision,
5     createtime timestamp without time zone,
6     ordernum character varying,
7     comment character varying,
8     paytype character varying,
9     goodstype character varying
10 );

```

The rest entities and relations are defined similarly. For more details, please consult the [benchmark specification](#) document and check the schema definitions in the provided `data.sql`.

[1] The LDBC Financial Benchmark (V 0.1.0). [Arxiv](#)

Submission

Make sure that you have PostgreSQL installed and have read the [Tips on using PostgreSQL](#). To set up the dataset, please download the file `data.sql` from the following address.

Download address: <https://jbox.sjtu.edu.cn//y10Fwe> (提取码: db2024)

After that, please use the command `\i data.sql` in the PostgreSQL interactive terminal. This will import both the database schema definition and the dataset. Please note that we are providing only a small dataset for test purposes. However, your submissions will be evaluated on a larger dataset, so it's important to write SQL queries that can handle various corner cases effectively.

Your assignment consists of five tasks. For each task, you need to write a SQL query and save it in a separate file. Name each file as `i.sql`, where `i` is the task number. For instance, your solution for Task 1 should be in a file named `1.sql`. Upon completing all tasks, compress all the SQL files and your solution to Problems 1&2 into a single ZIP file named `submission.zip` and upload it to Canvas.

```

1 ## After finish all your tasks
2 ## Please remember to include
3 ## (1) 1.sql // you will get 10 points for this.
4 ## (2) HW1.pdf // this file is your solution to Problems 1 & 2.
5 Assignment(I) → tree .
6 .
7 |— 1.sql
8 |— 2.sql
9 |— 3.sql
10 |— 4.sql
11 |— 5.sql
12 |— HW1.pdf
13 1 directory, 6 files
14
15 ## Create a zip file named submission.zip
16 Assignment(I) → zip submission.zip *.sql *.pdf
17 adding: 1.sql (stored 0%)
18 adding: 2.sql (stored 0%)
19 adding: 3.sql (stored 0%)
20 adding: 4.sql (stored 0%)
21 adding: 5.sql (stored 0%)
22 adding: HW1.pdf (stored 0%)
23
24 ## Now You can upload submission.zip to Canvas
25 Assignment(I) → tree .
26 .
27 |— 1.sql
28 |— 2.sql
29 |— 3.sql
30 |— 4.sql
31 |— 5.sql
32 |— HW1.pdf
33 |— submission.zip
34 1 directory, 7 files

```

Output Format

For the following 5 tasks, your output has two columns, `id` and `value`. Please read the tasks carefully for specific meaning, order, and format.

Task 1: Top 10 Destinations by Transfer Amounts (10 points)

Retrieve the top 10 accounts based on the sum of amounts transferred from other accounts. Here, `id` is the account ID and `value` is the transfer amount sum rounded to two decimal places.

A sample query `1.sql` for this task is shown below.

```
1 -- A sample 1.sql for task 1
2 -- Retrieve the top 10 accounts based on the sum of amounts transferred from others.
3 -- Output
4 -- (1) id: the account ID
5 -- (2) value: the transfer amount sum rounded to two decimal places.
6 SELECT a2.accountId as id, ROUND(CAST(SUM(t.amount) as numeric), 2) as value
7 FROM Account AS a1
8 JOIN AccountTransferAccount AS t ON a1.accountId = t.fromId
9 JOIN Account AS a2 ON t.toId = a2.accountId
10 GROUP BY a2.accountId
11 ORDER BY SUM(t.amount) DESC
12 LIMIT 10;
```

Execute this query in PG, you should be able to get the following output.

	id	value
1		
2		
3	4769031904798185087	2943820988.94
4	4761712455892076374	2860627681.03
5	4778321128785462589	2803343875.30
6	4752988106003595584	2779242376.02
7	171700010671408982	2622124701.73
8	252764803964086121	2486603851.80
9	4887533419749198229	2486186819.80
10	4777476703855322412	2484578341.73
11	4854318822741510055	2480129212.24
12	4805622827136858402	2400643947.90

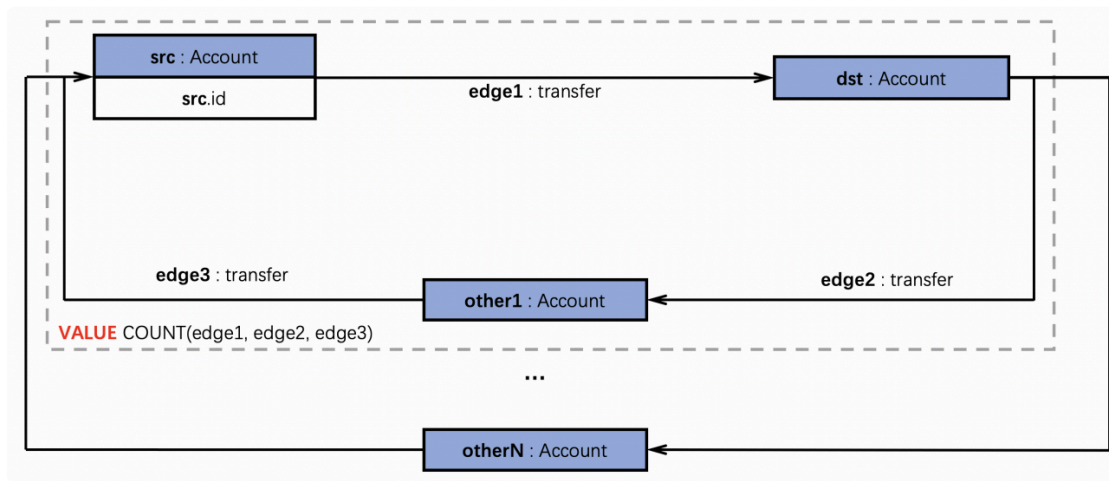
NOTE:

- If you can get the same output as shown above, it means that you have set up SQL Lab properly. Otherwise, please double-check your settings. You can contact our TAs if you need help.
- Please also include the sample `1.sql` in your submission and **get 10 points for free**.

Task 2: Transaction loop search (15 points)

Cycle detection is very useful in many financial applications. Please find every account `src` that matches the following conditions:

1. `src` transfers to the account `dst` via `edge1:transfer`.
2. `dst` transfers to another account `other` via `edge2:transfer`.
3. `other` transfers to `src` via `edge3:transfer`.



Closed transaction loop search

Here the `id` is the account ID of `src` that meets the conditions and the `value` is the number of corresponding transaction loops associated with `src`, i.e, the number of sequences `<edge1, edge2, edge3>`. Your output should also be sorted in *ascending order* by `id`.

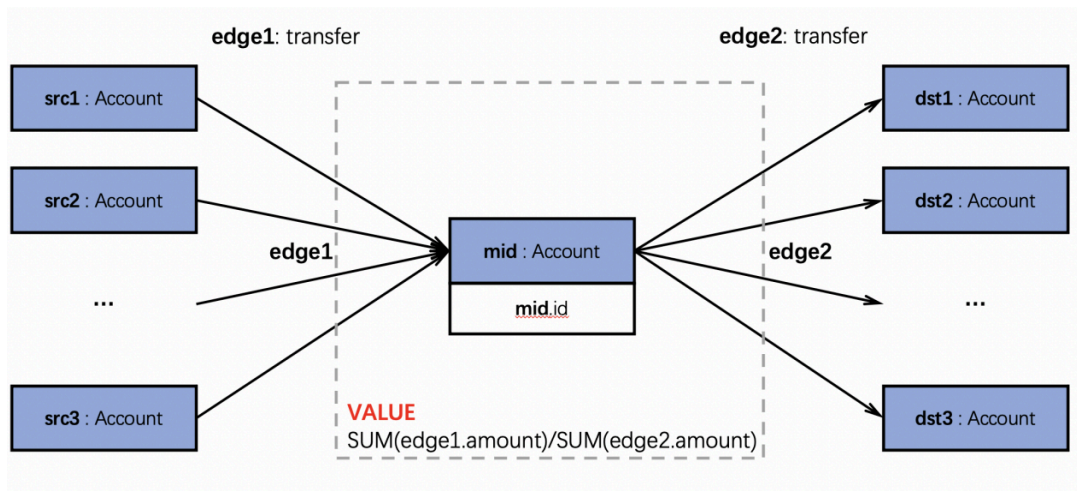
```

1 with RES as (
2   with TEMP as (
3     select E1.fromId as src, E1.toId as dst, E2.toId as other
4     from AccountTransferAccount as E1 join AccountTransferAccount as E2 on E1.toId
5 = E2.fromId
6     where E2.toId != E1.fromId
7   )
8   select TEMP.src, TEMP.dst, TEMP.other, E3.toId
9   from TEMP join AccountTransferAccount as E3 on TEMP.other = E3.fromId and TEMP.src
10 = E3.toId
11 )
12 select RES.src as id, count(toId) as value
13 from RES
   group by RES.src
   order by RES.src;
```

Task 3: Transfer in/out ratio (15 points)

Please find every account `mid` that meets the following conditions:

1. At least one account transfer to `mid` via `edge1:transfer`.
2. `Mid` transfer to at least another one account via `edge2:transfer`.



Fast in and out of funds

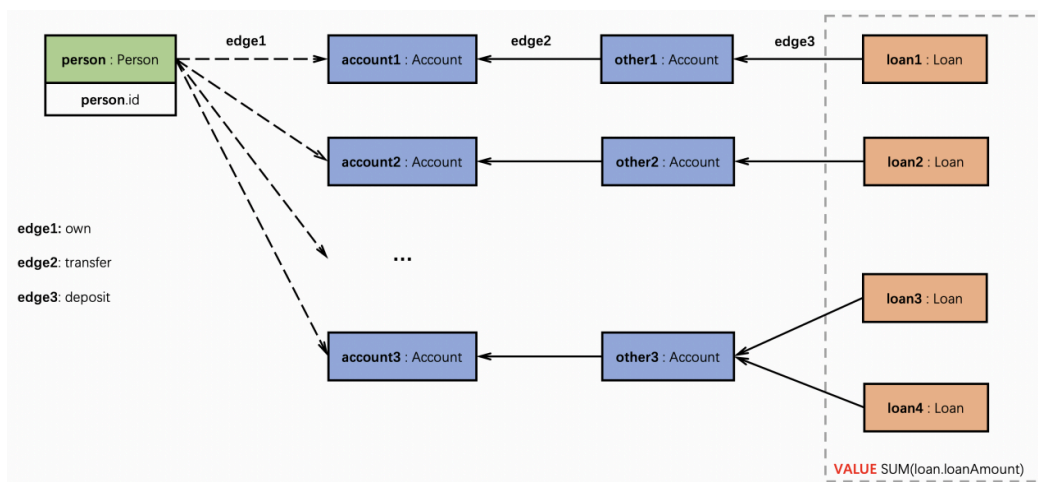
Here `id` represents the account ID of `mid` that meets the conditions, `value` is the **inflow and outflow ratio** of the transaction amount for the account `mid`, which can be computed as $\text{SUM}(\text{edge1.amount}) / \text{SUM}(\text{edge2.amount})$. The output should be sorted in ascending order by `id`, with each `value` rounded to two decimal places.

```
1 select id, ROUND(cast(cast(inflow as numeric) / cast(outflow as numeric) as numeric),2)
2 as value
3 from
4   (select toid, sum(amount) as inflow from AccountTransferAccount group by toid) as
5 E1
6   join
7   (select fromid as id, sum(amount) as outflow from AccountTransferAccount group
  by fromid) as E2
  on E1.toid = E2.id
 order by E2.id;
```

Task 4: Personal loan statistics (20 points)

Please find every `person` that meets the following conditions:

1. `Person` owns many accounts `account` via `edge1:own`.
2. An `account` receives money from another account `other` via `edge2:transfer`.
3. An `other` account deposits funds from at least one `loan` via `edge3:deposit`.



Personal loan statistics

For each `person`, please compute the amount sum of distinct loans as shown above. In the output, `id` represents the ID of `Person` and `value` equals `SUM(loan.loanAmount)`. The output should be also sorted in ascending order by `id`, with each `value` rounded to two decimal places.

```

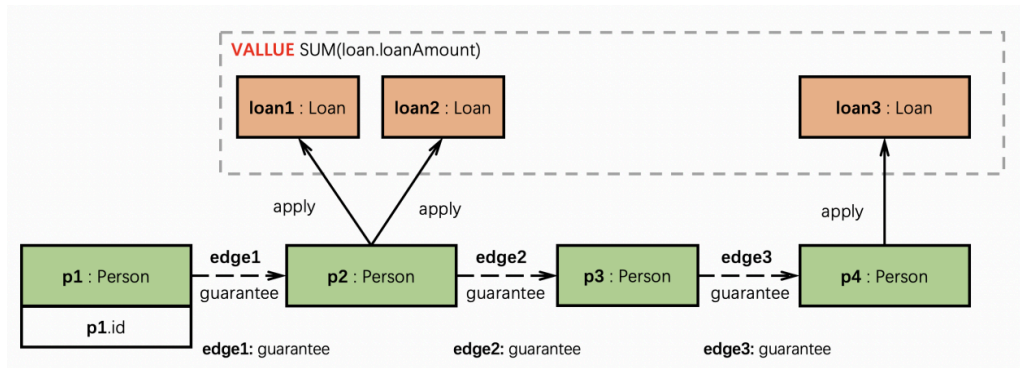
1 with RES as(
2     select distinct E3.personId, E1.loanId
3     from
4         LoanDepositAccount as E1
5         join AccountTransferAccount as E2 on E1.accountid = E2.fromId
6         join PersonOwnAccount as E3 on E3.accountId = E2.toId
7     order by
8         E3.personId
9 )
10 select RES.personid as id, ROUND(CAST(sum(loan.loanAmount) as numeric),2) as value
11 from RES join loan on RES.loanId = loan.loanId
12 group by personid
13 order by personid;

```

Task 5: Summary of guarantee amount (20 points)

Please find all the people `p1` who meet the following conditions:

- `p1` guarantees other users, and the guarantee chain is at least level 1 and at most level 3.
- Other users on the guarantee chain may apply for a loan, e.g., `p2` and `p4` in the figure below.



Summary of the guarantee amount

For each person `p1`, please summarize the loan amount in the guarantee chain. Here `id` represents the ID of `p1` that meets the conditions and `value` is the sum of loan amounts in `p1`'s guarantee chain. Please sort your output in ascending order by `id` and with each `value` rounded to two decimal places.

```

1 WITH RECURSIVE GuarantorChain AS (
2     SELECT PersonGuaranteePerson.fromId AS p1_id, PersonGuaranteePerson.toId AS p_id, 1
3     AS level
4     FROM PersonGuaranteePerson
5     UNION ALL
6     SELECT G.p1_id, PersonGuaranteePerson.toId AS p_id, G.level + 1
7     FROM GuarantorChain AS G
8     JOIN PersonGuaranteePerson ON G.p_id = PersonGuaranteePerson.fromId
9     WHERE G.level < 3
10 ),
11 RankdGuarantorChain AS (
12     SELECT p1_id, p_id, level,
13     ROW_NUMBER() OVER (PARTITION BY p1_id, p_id ORDER BY level) AS row_num
14     FROM GuarantorChain
15 ),

```

```
16 DeduplicatedGuarantorChain AS (  
17   SELECT p1_id, p_id, level  
18   FROM RankdGuarantorChain  
19   WHERE row_num = 1  
20 ),  
21 LoanSum AS (  
22   SELECT PersonApplyLoan.personId, SUM(Loan.loanAmount) AS total_loan_amount  
23   FROM PersonApplyLoan  
24   JOIN Loan ON PersonApplyLoan.loanId = Loan.loanId  
25   GROUP BY PersonApplyLoan.personId  
26 ),  
27 GuarantorLoanSum AS (  
28   SELECT G.p1_id, LS.total_loan_amount AS guaranteed_loan_amount  
29   FROM DeduplicatedGuarantorChain AS G  
30   JOIN LoanSum AS LS ON G.p_id = LS.personId  
31 ),  
32 TotalGuaranteedLoan AS (  
33   SELECT p1_id as id, ROUND(CAST(SUM(guaranteed_loan_amount) as numeric), 2) as value  
34   FROM GuarantorLoanSum  
35   GROUP BY p1_id  
36   ORDER BY p1_id  
37 )  
SELECT * FROM TotalGuaranteedLoan;
```